สรุปงาน Event tree และ Guard tree

198 แบบจำลอง · ตรวจกับแผนภาพต้นฉบับ 197 แบบจำลอง และกับไฟล์ Rodin 1 แบบจำลอง (model_001)

198	198 / 198	334	3,244
แบบจำลองที่มี tree	ผ่านการตรวจทุกข้อ	รหัส guard รวม	รหัส event รวม

1. งานนี้ทำอะไร

สกัด tree 2 แบบจากแบบจำลอง Event-B ทุกตัว โดยใช้ `model.txt` ที่ได้จาก ChronoSeq-B ตามเดิม ไม่แก้ไข (model_001 มีเฉพาะชื่อ event จึงอ่านจากไฟล์ proof obligation ของ Rodin แทน)

tree	node คืออะไร	ได้จาก
Event tree	event ของ Event-B (send และ receive แยกกัน) เรียงตามลำดับที่ทำงานได้	guard ควบคุมลำดับ เช่น `p ∈ receivedMessages`
Guard tree	เงื่อนไขจากแผนภาพ (loop / alt / opt) เรียงตามลำดับที่พบเมื่อระบบทำงาน	guard เงื่อนไข เช่น `mode = GOLD`

2. หลักการตั้งรหัส

	ตัวอักษร	ตัวเลข	ตัวอย่าง
guard	จุดตัดสินใจ: guard ที่นิพจน์ด้านซ้ายเหมือนกันใช้ตัวอักษรเดียวกัน	ทางเลือกของจุดนั้น	`mode = GOLD` b1 `mode = SILVER` b2
event	ลำดับขั้น: event ที่ต่อกันเป็นเส้นตรงได้ตัวอักษรใหม่ทีละขั้น	ทางเลือกตรงจุดแยกกิ่ง (alt หรือวนรอบใหม่ / ทำต่อ)	m1 / m2 / m3 = event แรกของแต่ละกิ่ง

event ที่หลายกิ่งมารวมกันได้ตัวอักษรใหม่ · ถ้าทางเลือกหนึ่งเป็นจุดรวม ทางเลือกอื่นตรงจุดแยกนั้นใช้ตัวอักษรของจุดรวม (รหัสจึงไม่ขึ้นกับลำดับที่อ่านกิ่ง)

ตัวอย่าง model_141 PharmacyDelivery (opt แล้วตามด้วย alt 3 ทาง)

Guard tree

```
(root)
└── a1  needAck = 1
    ├── b1  mode = GOLD
    ├── b2  mode = SILVER
    └── b3  mode = BRONZE
```

Event tree

```
(root)
└── a1 → b1 → c1 → d1 → e1 → f1 → g1 → h1 →
      i1 → j1 → k1 → l1
    ├── m1 → n1 → o1 → p1 → q1 → r1
    ├── m2 → n2 → o2 → p2 → q1 → r1
    └── m3 → n3 → q1 → r1
```

3. การตรวจสอบความถูกต้อง

วิธีตรวจ: อ่านแผนภาพ draw.io ต้นฉบับของแต่ละแบบจำลองด้วยตัวอ่านที่เขียนแยก ไม่ใช้โค้ดของตัวแปลงหรือตัวสกัด (อ่านกรอบ alt / opt / loop / par เส้นแบ่งกิ่ง ข้อความเงื่อนไข และข้อความตามตำแหน่งบนแผนภาพ) แล้วคำนวณว่า tree ต้องเป็นอย่างไร เทียบกับ tree ที่สกัดได้ 8 ข้อ (`tools/verify_trees.py`)

ข้อ	เรื่อง	สิ่งที่ตรวจ	ผล
C1	ข้อความ	ข้อความในแผนภาพตรงกับ event send / receive ใน Event-B ครบและเรียงลำดับเดียวกัน	ผ่าน 198 / 198
C2	guard รายevent	เงื่อนไขที่ครอบแต่ละข้อความในแผนภาพ ตรงกับ guard ของ event send และ receive ของข้อความนั้น	ผ่าน 197 / 197
C3	รหัส guard	มีรหัสครบทุกเงื่อนไขในแผนภาพ และไม่มีรหัสเกิน	ผ่าน 198 / 198
C4	ลำดับ event	event ก่อนหน้าของทุก event ตรงกับลำดับที่ได้จากแผนภาพ รวมจุดแยกกิ่งและจุดรวมของ alt	ผ่าน 197 / 197
C5	loop	event นับรอบ loop มีครบทุก loop มี guard ถูกต้อง อยู่ท้ายส่วนที่วน และย้อนกลับจุดเริ่มถูกต้อง	ผ่าน 197 / 197
C6	รหัส event	1 event มี 1 รหัส · ทางเลือกตรงจุดแยกกิ่งใช้ตัวอักษรเดียวกันและเลขไม่ซ้ำ · เส้นตรงไม่ใช้ตัวอักษรซ้ำ	ผ่าน 198 / 198
C7	เส้นทาง guard	เส้นทางใน guard tree ตรงกับทุกทางที่เป็นไปได้ในแผนภาพ	ผ่าน 197 / 197
C8	ไฟล์	ไฟล์ในชุดเทรน (models/, data/trees.jsonl, trees/guard_codes.csv) ตรงกับต้นฉบับทุกไฟล์	ผ่านทุกไฟล์

C2, C4, C5, C7 ตรวจได้เฉพาะแบบจำลองที่มีแผนภาพ (197 ตัว) · model_001 ตรวจ C1, C3 กับไฟล์ Rodin (.bpo) · ผลรายแบบจำลองอยู่ใน `check.csv`

ทดสอบตัวตรวจ

เพื่อยืนยันว่าตัวตรวจจับข้อผิดพลาดได้จริง จงใจทำ tree ที่ถูกต้องให้ผิด 10 แบบ แล้วให้ตัวตรวจตรวจซ้ำ · จับได้ **10 / 10** แบบ

ทำให้ผิดแบบ	แบบจำลอง	จับได้ที่ข้อ
ลบ guard ออกจาก event	model_109	C2
ข้อความ guard ผิด	model_109	C2
มีรหัส guard เกิน	model_109	C3
ตัดกิ่ง alt ออก	model_109	C4
loop ย้อนกลับผิดที่	model_109	C5
ทางเลือกใช้คนละตัวอักษร	model_109	C6
เส้นทาง guard หาย	model_109	C7
ลำดับ event ขาด	model_109	C4
ลำดับใน guard path ผิด	model_141	C7
เลขทางเลือกซ้ำ	model_141	C6

ปัญหาที่พบระหว่างตรวจ และการแก้ไข

ปัญหา	สาเหตุ	การแก้ไข
รหัส event ตรงจุดแยกกิ่งใช้คนละตัวอักษร (model_015, model_036, model_038)	ทางเลือกหนึ่งเป็นจุดรวมของ alt ก่อนหน้า (มีตัวอักษรของตัวเอง) อีกทางเลือกเป็น event นับรอบ loop จึงได้ตัวอักษรใหม่	ปรับกฎ: ทางเลือกอื่นใช้ตัวอักษรของจุดรวม (k1 / k2) · ตรวจซ้ำผ่านทุกแบบจำลอง

model_001 ไม่มี tree ในรอบแรก	model.txt มีเฉพาะชื่อ event	อ่าน event และ guard จากไฟล์ Rodin (.bpo)

4. ข้อสังเกตจากการตรวจ (พฤติกรรมของสูตรแปลง)

- **par** — สูตรแปลงเรียงข้อความใน par ตามลำดับบนลงล่าง ไม่มี guard ที่ทำให้ทำงานพร้อมกัน event tree จึงเป็นเส้นตรง
- **opt** — ข้อความหลัง opt ต้องรอข้อความสุดท้ายใน opt ใน Event-B จึงข้าม opt ไม่ได้ tree ไม่มีกิ่ง “ไม่เข้า opt”
- **loop** — ข้อความหลัง loop ต้องรอข้อความสุดท้ายใน loop จึงต้องทำอย่างน้อย 1 รอบ การวนรอบใหม่แสดงด้วย event นับรอบ (ʊ)
- **model_001** — แปลงด้วยสูตรรุ่นแรก loop และ par ไม่มี guard อ้างข้อความก่อนหน้า tree จึงแยกเป็น 2 กิ่งจาก root และมี 1 event ที่ไฟล์ Rodin ไม่เก็บชื่อ (unnamed_event_N)

ข้อสังเกตเหล่านี้เป็นคุณสมบัติของแบบจำลองที่แปลงได้ tree แสดงตามแบบจำลองจริง ไม่ได้ปรับให้ต่างจาก Event-B

5. สถิติ

จำนวนเส้นทางใน guard tree	**0**	**1**	**2**	**3**	**4**	**6**
จำนวนแบบจำลอง	52	71	67	5	2	1

รายการ	**ต่ำสุด**	**มัธยฐาน**	**สูงสุด**	**รวม**
event ต่อแบบจำลอง	8	15	30	3,244
เส้นทางใน event tree	1	2	6	387
รหัส guard ต่อแบบจำลอง	0	2	6	334
loop ต่อแบบจำลอง	0	0	3	79

แบบจำลองที่ไม่มีเงื่อนไขจากแผนภาพ 52 ตัว มีเฉพาะ event tree (guard tree ว่าง)

6. ไฟล์

```
EventB_PO_Training/
├── trees/
│   ├── SUMMARY.pdf          ← เอกสารนี้
│   ├── TREES.pdf            ← หลักการ ตัวอย่าง และรหัส + tree ของทุกแบบจำลอง
│   ├── guard_codes.csv      ← รหัส guard ทุกแบบจำลอง (model_id, code, guard)
│   ├── event_codes.csv      ← รหัส event ทุกแบบจำลอง (model_id, code, event, guards)
│   ├── tree_summary.csv     ← ตัวเลขต่อแบบจำลอง + เส้นทาง guard + ผลตรวจ
│   ├── check.csv            ← ผลตรวจรายแบบจำลอง
│   └── by_topic/<หัวข้อ>/<pass|fail>/<model>.txt   ← tree ของแต่ละแบบจำลอง (รูปย่อ + รูปเต็ม)
├── models/<id>/tree.txt · tree.json   ← ชุดเดียวกัน อยู่คู่กับ model.txt
└── data/trees.jsonl                   ← ทุกแบบจำลองในไฟล์เดียว (สำหรับโปรแกรม)
```

สร้างใหม่ทั้งหมดด้วย `python tools/make_release.py` **(สกัด → ตรวจ → สร้างเอกสาร) · ตัวสกัด** `tools/extract_trees.py` **· ตัวตรวจ** `tools/verify_trees.py`